DS GA 1011 Assignment 4: Fine-tuning Language Models

Fall 2026

1 Assignment Overview

In this assignment, you will train two types of language models: an encoder-only model (BERT) and an encoder-decoder model (T5). The main goal is learning how to fine-tune language models for a specific task, and understanding the challenges involved in doing so.

What you will implement

1. Fine-tuning of an encoder-only model (BERT) for sentiment classification, including a data transformation and data augmentation study of out-of-distribution generalization (§3)
2. Fine-tuning of an encoder-decoder model (T5) for text-to-SQL generation, a conditional generation task mapping a variable-length natural language input to a variable-length, structured SQL output (§4)

What you will run

1. Fine-tune BERT on a sentiment classification dataset, then evaluate it on a transformation of the test set designed to probe out-of-distribution generalization.
2. Fine-tune a model on the same transformation to perform data augmentation, and evaluate what this does and does not fix.
3. Fine-tune T5 to translate natural language flight-booking instructions into SQL queries, and evaluate it against a held-out test set.

What you can use This assignment requires GPUs. You should use standard Hugging Face `transformers` and PyTorch APIs for fine-tuning; see each part's `README.md` for environment setup.

Statement on AI tools Prompting LLMs such as ChatGPT is permitted for low-level programming questions or high-level conceptual questions about language models, but using it directly to solve the problem is prohibited. See the syllabus for more details.

We strongly encourage you to disable AI autocomplete (e.g., Cursor Tab, GitHub CoPilot) in your IDE when completing assignments (though non-AI autocomplete, e.g., autocompleting function names is totally fine). We have found that AI autocomplete makes it much harder to engage deeply with the content.

Where to find the code All the assignment code as well as this writeup are available on GitHub at:

`https://github.com/gregdurrett/nyu-dsga1011-a4`

Please `git clone` the repository. If there are any updates, we will notify you so you can `git pull`.

2 Submission

Submission is done on Gradescope. **Do not submit to Brightspace.**

Written A `.pdf` file must be submitted. We recommend using the released `.tex` file. In addition to your answers to the written questions, the PDF should include:

- A link to your GitHub repository containing your code for Part 1 and Part 2, and
- A link to a Google Drive folder containing the model checkpoint used to generate your outputs for Q7 (and for the extra credit submission, if applicable).

Programming You will submit the following output files for this assignment:

- Q1: `out_original.txt`
- Q2: `out_transformed.txt`
- Q3: `out_augmented_original.txt` and `out_augmented_transformed.txt`
- Q7: your model's SQL query outputs and the database records associated with them: `t5_ft_experiment_test.pkl` and `t5_ft_experiment_test.sql`
- Extra Credit (optional): `t5_ft_experiment_ec_test.pkl` and `t5_ft_experiment_ec_test.sql`

3 Part 1: Fine-tuning BERT for Sentiment Classification

In this part, we will explore fine-tuning a BERT model for a simple classification task. Our main goal is to explore the role of **data** in classification results, simulating out-of-distribution data.

Framework code You are provided with `main.py` and `utils.py` as starter code to fine-tune an encoder-only model on a sentiment classification dataset. `main.py` handles argument parsing, training, and evaluation. `utils.py` is where you will implement your data transformation in Q2.

3.1 Q1. Fine-tuning BERT

You will first write code to fine-tune a BERT model on the IMDB dataset for sentiment analysis.

Problem (`bert_finetuning`): Q1. Fine-tune BERT (10 points)

Deliverable: We have provided a template for running your code to fine-tune BERT, but it contains some missing parts. Complete the missing parts in the `do_train` function in `main.py`, which mainly includes running the training loop. You are not required to modify any hyperparameters.

Initial testing: Before proceeding, test your training loop on a small subset of the data

to verify its correctness. Use the following command: `python3 main.py --train --eval --debug_train`. Upon successful execution, you should achieve an accuracy of more than 88% on this small subset. (Estimated time: `7 min` train + `1 min` evaluation.)

Full training and evaluation: Once you have confirmed that your training loop is functioning as expected, fine-tune BERT on the full training data and evaluate its performance on the test data using the command: `python3 main.py --train --eval`. An accuracy of more than 91% on the test data is required to earn full points. (Estimated time: `40 min` train + `5 min` evaluation.)

Submission: Please submit the output file `out_original.txt` generated from the full training and evaluation step.

3.2 Q2. Data Transformations

The main objective in this part is creating a transformation of the dataset as out-of-distribution (OOD) data, and evaluating your fine-tuned model on this transformed dataset. The aim is to construct a transformation that is "reasonable" (e.g., something you could actually expect at test time) but not trivial. In most cases, the transformed example should have the *same label* as the original example — a human would assign the same label to the original and transformed example. For example, for an original movie review "Titanic is the best movie I have ever seen.", a transformation which maintains the label is "Titanic is the best film I have ever seen.".

Problem (`data_transformation_design`): Q2a. Design a data transformation (10 points)

Deliverable: Design a transformation of the dataset, and explain the details of what it does. You should describe it clearly, such that someone who reads the description can reimplement it. You should also include why that is a "reasonable" transformation, i.e., something which could potentially be an input at test time from a user. If you like, you can use LLMs to perform the data transformation.

Examples & guidelines Suitable transformations might include:

- Randomly replacing words in a sentence with their synonyms.
- Introducing typos into sentences.

An example of an **unreasonable** transformation is converting coherent words into random gibberish, such as changing "The soup was hot." to "The unnjk rxasqwer hot.". We've provided code guidelines for two transformations (synonym replacement and typo introduction). You can choose to utilize these or feel free to design a unique transformation of your own.

Evaluation criteria Determining whether a transformation is "reasonable" can be subjective. However, please exercise your best judgment. While we will be fairly flexible with the definition of "reasonable", extreme transformations like the gibberish example mentioned will not be accepted.

Problem (`data_transformation_impl`): Q2b. Implement the transformation (15 points)

Deliverable: Implement the transformation that you designed in Q2a.

Setup: We've provided an example transformation function named `example_transform`. Your task is to use this as a reference and complete the function `custom_transform` found in the `utils.py` file.

Debugging: To ensure that your transformation is working as expected and to see a few examples of the transformed text, use the following command: `python3 main.py --eval_transformed --debug_transformation`

Evaluation: To assess the performance of the trained model on your transformed test set, execute: `python3 main.py --eval_transformed` (Estimated time: `5 min` evaluation.)

Your grade will be determined based on the decrease in performance on the transformed test set in comparison to the original test set:

- A decrease in accuracy of up to 4 points will grant you partial credit (8 out of the total 15 points).
- A decrease in accuracy of more than 4 points will award you the full 15 points.

Submission: Please submit the output file `out_transformed.txt` generated from the evaluation step.

3.3 Q3. Data Augmentation

In this part, you will develop a simple way to improve performance on the transformed test set, according to the transformation you defined in Q2. One simple way to potentially improve performance is to apply a similar transformation to the training data, and train your model on a combination of the original data and transformed training data. This method is usually known as data augmentation. You will augment the original training data with 5,000 random transformed examples to create a new augmented training dataset. Fill in `create_augmented_dataloader` in `main.py` to complete the code.

Training & evaluation Train a model using this augmented data by executing the following command: `python3 main.py --train_augmented --eval_transformed` (Estimated time: `50 min` train + `5 min` eval.)

Problem (`data_augmentation`): Q3. Data augmentation (15 points)

Deliverable: Evaluate the performance of the model trained above. Your objective is to assess how data augmentation affects the model's ability to handle both original and transformed test data.

Evaluation on original test data: Execute the following command: `python3 main.py --eval --model_dir out_augmented` (Estimated time: `5 min` eval.)

Evaluation on transformed test data: Execute the following command: `python3 main.py --eval_transformed --model_dir out_augmented` (Estimated time: `5 min` eval.)

Report & analysis:

- Report the accuracy values for both the original and transformed test data evaluations.
- Analyze and discuss: (1) Did the model's performance on the transformed test data improve after applying data augmentation? (2) How did data augmentation affect the model's performance on the original test data? Did it enhance or diminish its accuracy?
- Offer an intuitive explanation for the observed results, considering the impact of data augmentation on model training.
- Explain one limitation of the data augmentation approach used here to improve performance on out-of-distribution (OOD) test sets.

Submission: Please submit `out_augmented_original.txt` and `out_augmented_transformed.txt`, obtained after running model evaluation on both test sets.

4 Part 2: Fine-tuning T5 for Text-to-SQL

Now you will be working with the small variant of the T5 encoder-decoder architecture (Raffel *et al.*, 2019)[1]. The encoder will ingest natural language queries (i.e., the input sequence) while the decoder will predict the corresponding SQL queries (i.e., the output sequence). You will finetune the **pretrained T5 architecture**. The grading for this part of the homework consists of a **written part (25pt)** as well as the **performance (25pt)** of your trained model.

Framework code & data The data can be found under the `data/` directory. The database you will be evaluating queries on is `flight_database.db`, with the `flight_database.schema` file containing the database schema. The `ents` (entities) section in the schema lists the 25 tables in the database, such as "airline", "restriction", "flight", etc. The schema also gives information about the columns in the table, such as their type and whether they're indexed.

The text-to-SQL data is split into training, development, and held-out test sets. Files with the `.nl` extension contain the natural language instructions, while files with the `.sql` extension contain the corresponding, ground-truth SQL queries. The starter code contains utility functions for evaluation (under `utils.py`) that abstract away the details of interacting with the SQL database. A skeleton of training code for T5 is provided in `train_t5.py`.

We provide you a training loop code skeleton where you will only need to implement model initialization (using the relevant Hugging Face `transformers` implementation), the evaluation loop, and data processing. You should initialize the model with the following checkpoint: `https://huggingface.co/google-t5/t5-small`.

Simply implementing data processing with the existing T5 tokenizer and varying simple hyperparameters should lead to good baseline results. You may, however, choose to experiment with data processing or architecture details to push performance higher. During finetuning, for instance, a common design choice is to freeze part of the model's parameters (i.e., only finetune a subset of the parameters). In the assignment report, we ask about your strategies for data processing, tokenization, and architecture details (such as freezing parameters). If you experiment with these

[1] `https://arxiv.org/pdf/1910.10683`

dimensions and find them to improve performance, we will expect ablation experiments supporting your findings in the results section.

Task evaluation details As different SQL queries can correctly respond to the same natural language instruction, evaluation is commonly performed by executing the SQL queries directly on the database. The evaluation code in the starter repository provides three separate metrics: Record F1, Record Exact Match, and SQL Query Exact Match (EM).

Record F1 is the metric we will use for the leaderboard; it computes the F1 score between database records produced by model-generated and ground-truth SQL queries. Record EM is a stricter metric checking whether the produced records match exactly, similar to an accuracy score. SQL Query EM tells you whether the SQL queries themselves match exactly. As the latter two can help with debugging and error analysis, we will also ask you to report them on the development set in the results section.

To verify the correctness of your output format, you may use the `evaluate.py` script. For instance, to evaluate submission files on the dev set, run:

```
python evaluate.py
  --predicted_sql results/t5_ft_dev.sql
  --predicted_records records/t5_ft_dev.pkl
  --development_sql data/dev.sql
  --development_records records/ground_truth_dev.pkl
```

Problem (`data_stats`): Q4. Data statistics and processing (5 points)

Deliverable: Describe the data statistics before and after any pre-processing by filling in Table 1 and Table 2. Use the T5 tokenizer to report the statistics. The gray text in each row is there to guide you and should be removed in your submitted report. Depending on your pre-processing, some numbers may be identical across tables.

Statistics Name	Train	Dev
Number of examples	XXX	XXX
Mean sentence length	XXX	XXX
Mean SQL query length	XXX	XXX
Vocabulary size (natural language)	XXX	XXX
Vocabulary size (SQL)	XXX	XXX

Table 1: Data statistics before any pre-processing. You need to at least provide the statistics listed above, and can add new entries.

Problem (`t5_finetuning_details`): Q5. T5 fine-tuning details (10 points)

Deliverable: Use Table 3 to describe your data processing steps (if any) and implementation details for the fine-tuned T5 model. The gray text in each row is there to guide you and should be removed in the submitted report. Be clear enough that one can replicate your

Statistics Name	Train	Dev
Model name		
Statistics Name	XXX	XXX

Table 2: Data statistics after pre-processing. You need to at least provide the statistics listed in Table 1 (except for the number of examples), and can add new entries.

approach in PyTorch using only your descriptions.

Design choice	Description
Data processing	Describe the data processing steps you undertook, if any.
Tokenization	Describe how you did the tokenization for the inputs to the encoder and decoder. If you use anything else than the default T5 tokenizer, specify what you use, and why you choose to use it.
Architecture	Describe the components in the T5 architecture that you chose to fine-tune. Did you fine-tune the entire model, specific layers?
Hyperparameters	List the key hyperparameters that you used, including the learning rate, batch size, and stopping criteria.

Table 3: Details of the best-performing T5 model configuration (fine-tuned).

Problem (`t5_results`): Q6. Results and error analysis (10 points)

Deliverable: **Quantitative results**: Use Table 4 to report your test and development set results. Your test results should match the results on the leaderboard. For the development set, also report results from experiments you conducted to arrive at your final configuration. "Full model" refers to the best model you described in Q5. You can add rows specifying variants you tried. The text in gray is only for example purposes and should be removed and replaced with your own results.

Qualitative error analysis: Conduct a detailed error analysis for your model. Identify common error types and discuss possible reasons for these errors in Table 5. You must identify at least three classes of errors for the queries, and use examples to illustrate them. If you identified the same type of error for different models (e.g., if you also did an extra credit model), you don't need to duplicate the descriptions, but you need to clearly specify an example for each model and indicate the statistics for each. You may add more rows to the table.

Problem (`t5_performance`): Q7. Performance evaluation (25 points)

Deliverable: **Submission**: We will maintain a leaderboard for the test set, with sub-

System	Query EM	F1 score
Dev Results		
T5 fine-tuned		
Full model	XX.XX	XX.XX
Test Results		
T5 fine-tuning	XX.XX	XX.XX

Table 4: Development and test results. Use this table to report quantitative results for both dev and test results.

Error Type	**Example Of Error**	**Error Description**	**Statistics**
Error name	Snippet from datapoint exemplifying error	Describe the error in natural language	Provide statistics in the form "COUNT/TOTAL" on the prevalence of the error. TOTAL is the number of relevant examples (e.g. number of queries, for query-level error), and COUNT is the number of examples that showed this error.

Table 5: Use this table for your qualitative analysis on the dev set.

missions ranked according to F1 score. As it can be resource intensive to run SQL queries, you will submit two files: `t5_ft_experiment_test.sql`, the model's SQL query outputs; and `t5_ft_experiment_test.pkl`, the database records associated with the SQL queries.

Performance evaluation criteria: To get full credit for performance, your model should achieve ≥ 65 F1 on **the test set**. While you can submit multiple times to Gradescope to get a test set result, you should not use this functionality to hill-climb on the test set; develop your model by checking performance on the provided development set, and run on the test set only a few times (ideally once). We may deduct points if you run on the test set excessively.

If your model performs below this threshold, you'll receive partial credit, computed as $\frac{\text{your score}}{65} \times 25$.

The `save_queries_and_records` function in `utils.py` will help you produce the submission files. Please refer to the `README.md` in the code skeleton for more details on formatting.

A few tips

- Start early.
- Start from a simple baseline. Then move on to fine-tuning and other modifications. This is a better approach as it will help you debug faster.
- If you are using AI tools, do not give the entire PDF and entire codebase to Claude Code/Codex and ask it to generate the code. This is less likely to work.
- For faster debugging, use your local machine for 1-2 iterations and then submit a SLURM

job on HPC. You may also use Lightning.ai. You can also use a Jupyter Notebook on HPC for debugging, but it may be revoked.

- Create multiple overlay files in HPC and submit parallel jobs. This will help you iterate faster.
- If you are seeing a reduction in loss with F1 score plateauing, you may try the following:
  - Try changing the learning rate.
  - Ensure that you have used the EOS, BOS, or other special tokens (if any) correctly.
  - Ensure that you have used the right sampling method(s).

Extra Credit

You can complete the extra credit assignment below for 10% extra credit on this assignment.

Problem (`extra_credit_scratch`): Extra Credit: Training T5 from scratch

Deliverable: In this optional task, you will train the exact same architecture **from scratch (i.e., from randomly initialized weights)**. This is more challenging than fine-tuning pretrained weights, and performance will be lower. Yet, you should be able to achieve decent results with the existing T5 tokenizer and varying hyperparameters. You could also find that experimenting with data processing leads to better outcomes. If you find that the tokenizer is ill-suited for SQL code, for instance, you could choose to design your own tokenizer for SQL and learn new embeddings and language modeling output heads for the decoder.[a] Describe your new model in the report (i.e., make a new version of Table 1–Table 5 with your new model and clearly label them).

To receive the extra credit, you should achieve ≥ 50 F1 with this model on the hidden test set, along with valid descriptions.

Please submit `t5_ft_experiment_ec_test.pkl` and `t5_ft_experiment_ec_test.sql` for extra credit.

[a] These are ideas to get you thinking, rather than specific suggestions to try. We did not fully experiment with these approaches.

Acknowledgement: Part 1 was originally developed by He He for previous offerings of this course, developed based on "NL-Augmenter," which can be accessed at `https://arxiv.org/abs/2112.02721`. Part 2 was modified from an assignment from LM-class.org, created by Omer Gul, Anne Wu, and Yoav Artzi.